MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Expert offloading enables single-GPU inference for mixture-of-experts (MoE) models. Repeated HDD reads make expert residency a source of inference latency. We present MECC, the MoE Expert Cache Compiler, for profile-guided placement and joint HBM/CPU caching. Its compiler allocates slots by layer and emits the expert IDs used for initialization. The runtime promotes router-selected experts and retains displaced GPU weights in the host cache. The delivery path orders HDD reads and H2D copies before GPU consumption. On one A100, MECC decodes FP16 Mixtral-8x7B at 7.38 s/token with uniform initialization. Decode speedups are 1.77× over streaming and 2.95× over fixed CPU retention, with matching perplexity.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models scale parameter capacity through sparse expert activation [1]–[4]. For example, Mixtral selects two of eight experts in each layer. A single-GPU deployment can use high-bandwidth memory (HBM), host memory, and a hard disk drive (HDD) to hold that pool. The selected expert's location then determines its delivery path. HBM residents are ready for GPU execution; host residents require a host-to-device (H2D) copy; cold experts require an HDD read followed by H2D. The next layer consumes the current layer's output, so an exposed read also delays downstream computation. These accesses recur at successive MoE layers during generation. Prefill can reuse one fetched expert across prompt rows, and decoding can reuse its weights across output tokens. Retaining the corresponding cache state avoids fetching those weights again. The placement problem is therefore tied to the request sequence, rather than to model loading alone. On shared servers, a serving task manages its expert allocation alongside other jobs and model instances. We study how to distribute that allocation across layers and preserve the resulting reuse during inference. All MoE layers share the HBM and host allowances. Increasing one layer's allocation leaves fewer slots for others. The value of another slot depends on the demand already covered in that layer. Placement therefore couples the layers' local request distributions to a shared storage allocation.

Expert parallelism places experts across GPUs [5]–[7]. Single-GPU systems instead combine caching, transfers, and host execution [8]–[12]. MoE-CORE uses layer-wise allocation, profile initialization, and online replacement [13]; SSD-LLaMA coordinates RAM/VRAM retention with storage delivery [14]. Edge FlashMoE studies cache replacement for storage-backed expert access [15]. Within such a hierarchy, initial placement and later admission affect different requests. A profile determines which weights are available when serving begins. GPU turnover then determines which of those weights survive on the host. Promoting a CPU expert vacates a host position just as an HBM victim loses its GPU position. Using that position for the victim preserves its place in the reuse order. Figure 1 compares this joint rule with independent HBM/CPU caches on an A100. Decode time is 7.38 s/token under joint retention and 32.94 s/token under independent replacement. Their HDD-demand shares are 12.5% and 47.9%. These results motivate compiling an initial resident set and preserving subsequent reuse across both tiers.

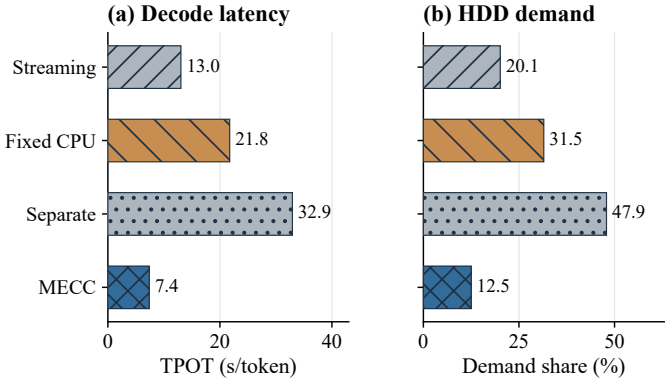


Fig. 1. Single-A100 expert-cache comparison on Mixtral-8x7B. Separate uses independent HBM/CPU replacement. Joint retention lowers both plotted metrics.

We present MECC, the MoE Expert Cache Compiler, to connect these two stages. Compilation ranks experts within each layer and allocates slots by marginal covered demand. The resulting plan specifies both per-layer capacities and the IDs loaded into HBM and CPU. The runtime then maintains a joint recency order across those tiers. A CPU hit promotes the selected expert and demotes the GPU victim; a cold admission can also evict a host resident. The runtime keeps a host mirror with each HBM resident and reuses that copy during demotion. Host admission therefore updates the cache metadata at that boundary. Newly requested experts follow the read/copy path, and execution waits for their weights. Consuming-stream tracking keeps the tensor storage live through kernel execution. This process keeps the capacities fixed while updating membership from the actual request stream. The profile supplies an initial distribution, and runtime accesses supply an order of use. Weight delivery follows the selected experts' read and copy dependencies before GPU execution. Prefill submits cold reads for the active expert batch, while decode loads selected experts on demand. The same layer/expert identity passes from profiling through placement to execution, preserving the router's assignments and FP16 expert computation.

This paper makes three contributions.

(1) **Compiling initial residency.** We map layer profiles and expert allowances to per-layer capacities and ranked expert IDs. A marginal-coverage scan determines how each additional slot extends the initial resident set.

(2) **Updating cross-tier residency.** We connect the compiled state to joint promotion and victim demotion. Host retention preserves GPU evictions for later reuse, while read/copy ordering prepares selected weights for consumption.

(3) **Evaluating weight delivery.** We compare placement, retention, and scheduling through decode latency, first-token time, source-tier demand, and sample perplexity. Workload and capacity sweeps examine calibration, task transfer, prompt length, batch size, and host allocation. Deployment comparisons report latency and accelerator occupancy.

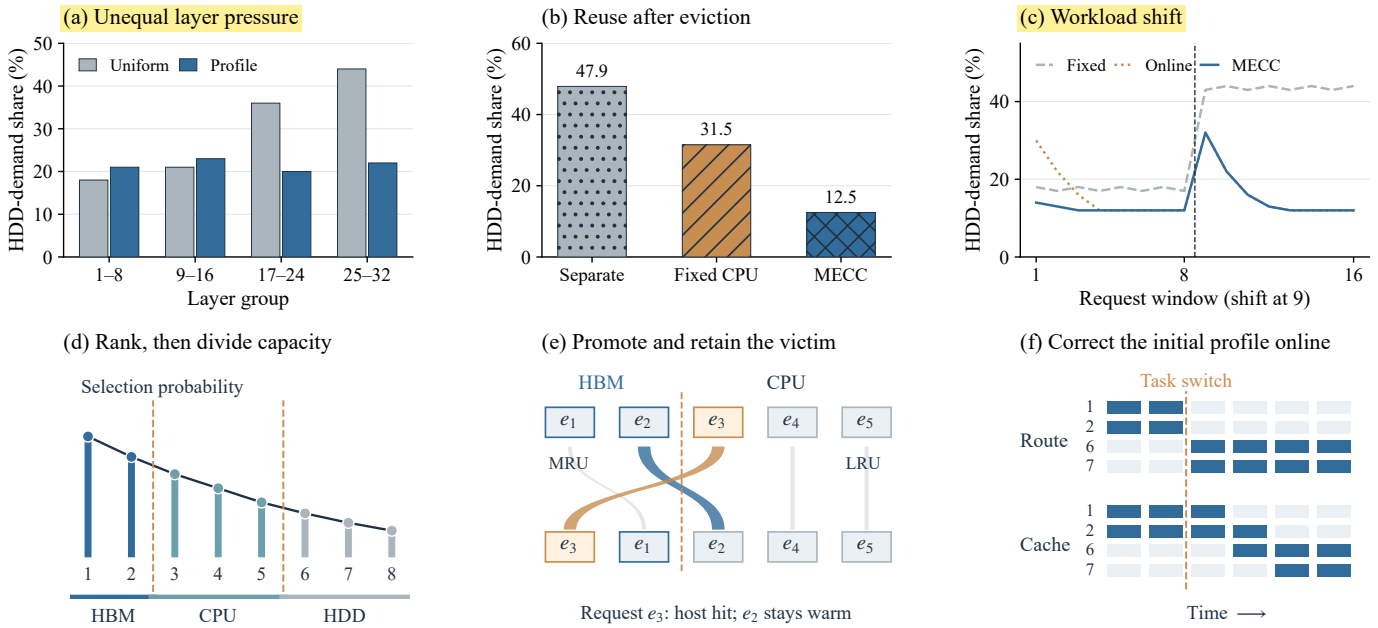


Fig. 2. Motivation evidence and matching insights. (a–c) compare layer pressure, cross-tier reuse, and workload change; (b) counts A100 demand across all phases. (d–f) show layer-specific capacity cuts, joint victim retention, and online membership updates.

II. Background

A. Sparse Execution, Full Weight Capacity

An MoE layer routes each token to a subset of experts and combines their outputs [1], [2], [16]. Mixtral and the DeepSeek family separate active computation from the size of the stored expert pool [3], [4], [17]–[19]. An expert belongs to one layer; equal indices at different layers identify different weights. For input x, let $G(x)$ be the selected set and $g_e(x)$ its normalized gate weights. The layer computes

$$y(x) = \sum_{e \in G(x)} g_e(x) E_e(x). \tag{1}$$

A token traverses the MoE layers in order, revealing expert demand at each gate. Decoding revisits those layers for the next token. Caching changes the source of the selected weights while preserving the routing weights and expert functions.

B. Expert Residency on One GPU

Expert parallelism places experts on GPU owners and dispatches token activations to them [5]–[7], [20], [21]. A single-GPU offloading system instead delivers selected weights to the execution device. Experts share HBM with dense weights, activations, and the KV cache [22], [23]. The host cache supplies H2D copies; HDD supplies weights absent from both resident tiers. Heterogeneous inference systems combine these paths with CPU/GPU scheduling [24]–[28]. In prefill, several prompt rows can share a fetched expert. Decode advances one token per sequence, exposing fewer rows to each active expert. The amount of ready computation therefore changes between the two phases. MECC retains FP16 expert weights across these layer visits and uses HDD as the backing store.

A cache entry identifies one expert's weights and its position in the resident order. HBM and host residency persist across layer calls. A later router selection can therefore reuse weights loaded by an earlier token. Calibration counts determine expert ranking; runtime accesses determine recency. Within one layer, sequences $[a, a, b, b]$ and $[b, b, a, a]$ have equal counts and different last-access orders. MECC uses both kinds of information.

III. Motivation and Insights

A. Allocating Slots by Marginal Demand

Equal expert counts do not imply equal demand across layers. A few ranks may cover most requests in one layer, while another layer spreads requests over more experts. Figure 2(a,d) illustrates the resulting allocation choice. Its profile-guided plan shifts slots toward the layer groups with more cold demand. The compiler makes this choice by comparing each layer's next uncovered rank. Allocating a slot exposes that layer's following rank as a new candidate. This process distributes a shared allowance across layers and retains the IDs that fill each allocation. Routing locality across models has also been studied independently [29].

B. Preserving Reuse after GPU Eviction

A GPU victim is also a candidate for host admission. Figure 2(b,e) shows a CPU promotion vacating a host position as the GPU selects its victim. Joint retention admits that victim into the vacated position, carrying recency across the tier boundary. The independent-cache control reports 47.9% HDD demand, compared with 12.5% for the joint rule. Both start from the same allocation and expert order. The difference concerns how each policy retains the weights selected during execution. Preserving the victim lets a later selection use its host copy instead of initiating another cold read.

C. Updating Residency beyond Calibration

Calibration and serving inputs can select different expert sets. In Figure 2(c,f), a task switch changes demand while preserving the cache capacities. The fixed placement continues to retain its original expert set. Online admission instead brings newly selected experts into the resident order. Profile initialization lowers the early demand transient, and online replacement follows the subsequent switch. These effects occur at different points in the same request sequence. MECC therefore initializes each resident set from the calibration profile. Runtime accesses then update its membership within the compiled per-layer capacities.

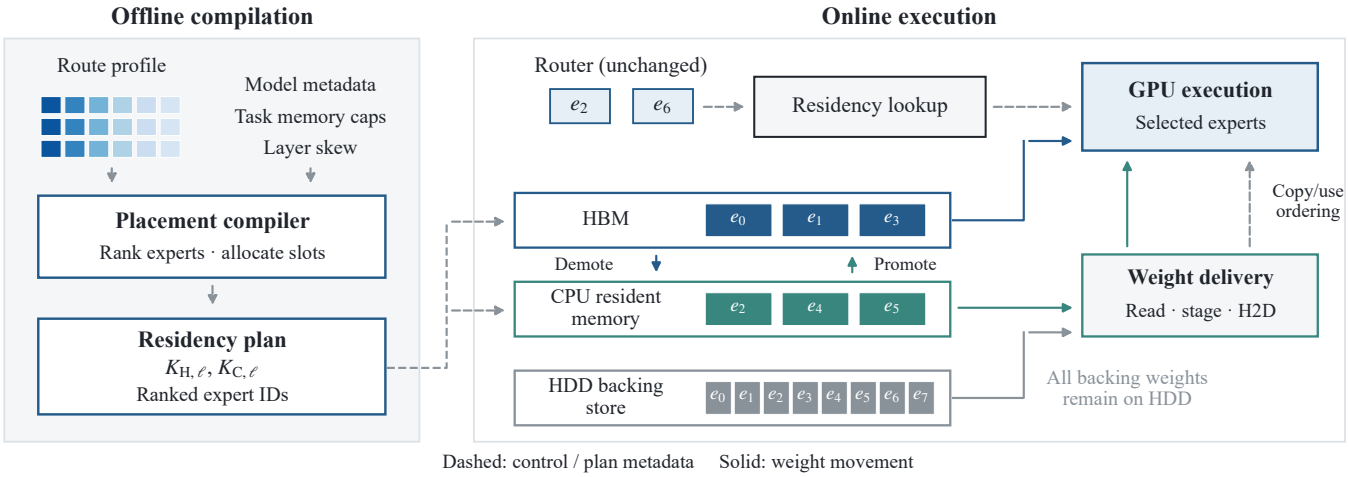


Fig. 3. MECC framework. The placement compiler emits per-layer capacities and ranked IDs. Online execution resolves unchanged router selections against HBM, CPU resident memory, and HDD, then delivers weights before their GPU consumers. Dashed arrows carry control or plan metadata; solid arrows move weights.

IV. Overview

Figure 3 follows the plan from profiling to GPU execution. The compiler receives layer/expert counts, skew parameters, model metadata, and expert allowances. It emits capacities and ranked IDs for each layer. Initialization loads the corresponding HBM and CPU residents; the complete expert pool remains on HDD. The runtime subsequently consumes this plan as an initial cache state.

Consider the selected experts e_2 and e_6 in the framework. The residency lookup finds e_2 on the host and e_6 in the backing store. Promoting e_2 frees a host position for an HBM victim. Loading e_6 then admits another GPU resident and updates the same tier order. The delivery path resolves each weight's read and copy before its consumer executes. The model combines the expert outputs using the gate weights and continues to the next layer.

After a layer finishes, its residents remain available for the next token. Capacities still come from the compiled plan, but recency reflects the accesses just served. Each layer keeps its own expert identities and resident state. This separates the profile's distributional information from the request order encountered during generation. Section V details the allocation and transition rules; Section VI examines their effect on weight delivery.

V. Method

A. Compiling Capacities and Initial Identities

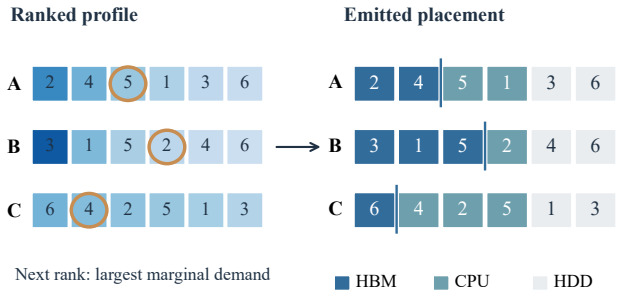


Fig. 4. Compiling the initial state. The left grid contains ranked per-layer demand; circles mark the next uncovered candidates. Shared HBM/CPU slot allowances produce different cuts in the emitted rows. A, B, and C identify different layers.

Consider L MoE layers with N_ℓ experts per layer. Within each model, experts have equal size s and layers use the same top-k. Expert allowances B_H and B_C provide $S_H = \lfloor B_H/s \rfloor$ HBM slots and $S_C = \lfloor B_C/s \rfloor$ CPU slots. These slots define the primary resident sets.

For each layer, the compiler receives an expert order and a skew parameter α_ℓ. Profile counts determine the order. A discrete Zipf model gives the probability associated with each rank:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{\sum_{j=1}^{N_\ell} j^{-\alpha_\ell}}, \qquad p_{\ell,r} = q_{\ell,r}/L. \tag{2}$$

Here $q_{\ell,r}$ is conditional on an access to layer ℓ. Equal top-k gives equal selection mass across layers. The ranked list preserves the original ID at each position.

Figure 4 shows the HBM and CPU cuts in those lists. Capacities satisfy $\sum_\ell K_{H,\ell} \le S_H$, $\sum_\ell K_{C,\ell} \le S_C$, and $K_{H,\ell} + K_{C,\ell} \le N_\ell$. The adaptive scan reserves one slot per layer in each fast tier, then distributes the remaining slots by marginal coverage.

The scan first assigns HBM slots. At each step, it chooses the layer with the largest next-rank score:

$$\Delta_\ell = q_{\ell,K_{H,\ell}+1}. \tag{3}$$

CPU allocation continues from the first ranks beyond each HBM cut. Every allocation exposes a new next-rank candidate in that layer. Layer ID breaks score ties, and expert ID breaks count ties. The direct scan takes $O(L(S_H + S_C))$ time after sorting. Its score is the request probability covered by the next slot.

The emitted plan contains the final capacities and ordered expert IDs. Initialization places each layer's first HBM ranks on the GPU and its next CPU ranks on the host. Natural ID order initializes the uniform control. During execution, admission changes membership within the compiled capacities.

B. Retaining Victims across the Tier Boundary

Each layer maintains two ordered sets, from most-recently used (MRU) to least-recently used (LRU). Their concatenation places HBM residents before CPU residents. An HBM hit moves its expert to HBM MRU. A CPU hit removes its host entry and promotes it to HBM MRU. The GPU LRU victim then becomes CPU MRU. A cold admission follows the same GPU insertion and can remove the CPU LRU. HDD retains the backing weights for every expert.

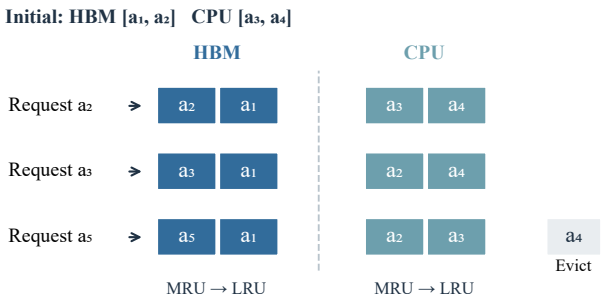


Fig. 5. Three requests from the same initial state. A hit changes recency; a host promotion also demotes the GPU victim; a cold admission can additionally evict the CPU victim. Rows show completed logical states, each ordered MRU to LRU.

Each row in Figure 5 starts from HBM $[a_1, a_2]$ and CPU $[a_3, a_4]$. An a_2 hit changes the GPU order. Promoting a_3 releases its CPU position, which receives the HBM victim a_2. A cold a_5 admission releases no CPU position. Retaining its GPU victim therefore removes the host LRU a_4. The two admission paths use the same demotion rule; their difference is the host space released by the request.

Completed serial transitions implement exclusive LRU over the combined resident capacity. Batch preparation can schedule several promotions before their consumers execute. Temporary copies bridge that preparation to the same ordered expert computation. The initial plan supplies the first resident order; subsequent requests update it.

A host mirror accompanies each GPU resident and supplies its weights during demotion. Reusing that copy avoids a return transfer from the GPU. Cache eviction changes membership, while stream tracking keeps the underlying storage live through GPU consumption.

C. Ordering Delivery before Expert Consumption

Prefill groups token rows by selected expert and submits the corresponding cold reads. Each read is resolved when its expert reaches consumption. Figure 6 shows resident work overlapping a pending read for another selected expert. Decode uses demand delivery. Both paths preserve the router's selected set and aggregation order.

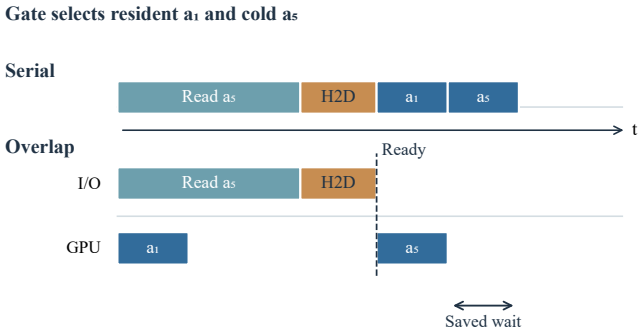


Fig. 6. Prefill delivery of resident a_1 and cold a_5 selected by one gate. Overlap executes ready work during the same read/copy path. The copy/use boundary orders a_5's consumer; the host source remains live through copy completion.

A completed read future supplies the host weights for H2D. The loader waits on the copy stream before yielding CUDA weights to a consumer. Recording that consumer's stream preserves tensor storage until its kernels finish. Read completion, copy synchronization, and storage lifetime therefore order the same selected expert from HDD to execution.

The available overlap comes from work ready to execute during the read batch. Once a transfer completes, the next selected expert can consume its weights. Retention and scheduling affect different terms: retention avoids a delivery, whereas overlap shortens its exposed time. The component comparisons examine both effects.

D. Sharing the Expert Execution Interface

All policies use the same FP16 loader, model graph, and router. Layer/expert IDs resolve to weight tensors, while the router supplies the selected set and aggregation weights. A policy chooses residency and the source tier for each request. Delivery then prepares those tensors for the shared expert consumer.

VI. Evaluation

A. Evaluation Overview

We report time per output token (TPOT), time to first token (TTFT), expert-demand classifications, and workload sensitivity. Placement, retention, and scheduling comparisons follow the corresponding method decisions. Deployment results report latency alongside accelerator occupancy. Appendix A gives configurations, policy definitions, and figure data annotations.

B. Single-GPU Performance

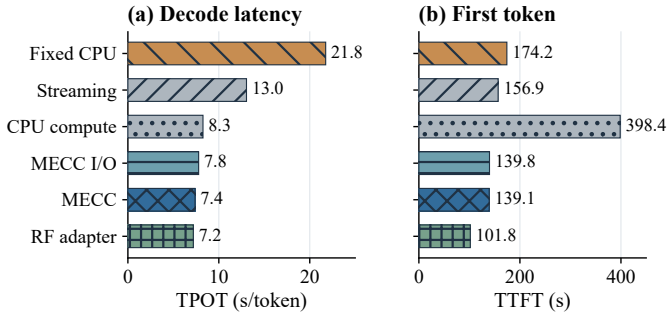


Fig. 7. Single-A100 runtime comparison. The panels report per-token decode time and time to first token.

Figure 7 reports 7.38 s/token for MECC, compared with 13.04 for streaming and 21.75 for fixed CPU retention. The corresponding decode speedups are 1.77× and 2.95×. The recency/frequency (RF) adapter, MECC I/O, and CPU-compute take 7.19, 7.78, and 8.26 s/token. First-token time is 139.1 s for MECC, 156.9 for streaming, and 174.2 for fixed CPU. Perplexity (PPL) is 3.427 for the GPU-execution policies and 3.432 for CPU-compute. The timing panels capture the prompt's first response and the recurring cost of later tokens. Weight residency persists between these phases, allowing decode requests to use experts admitted during earlier layer visits.

The reported access mix also distinguishes the MECC and RF policies. MECC records 77.5% HBM, 10.0% CPU, and 12.5% HDD; RF records 52.2%, 36.6%, and 11.1%, respectively. MECC thus has a larger HBM-hit component, while RF has a larger host-hit component. Both execute the selected FP16 experts through the shared delivery interface. Their differences lie in which experts remain resident after each admission. A GPU hit supplies an expert already available for consumption; a host hit supplies weights through H2D. RF combines frequency with recency, whereas MECC maintains a joint recency order. These rules can retain different host experts after admitting the same GPU victim. The recorded demand and latency follow those residency decisions.

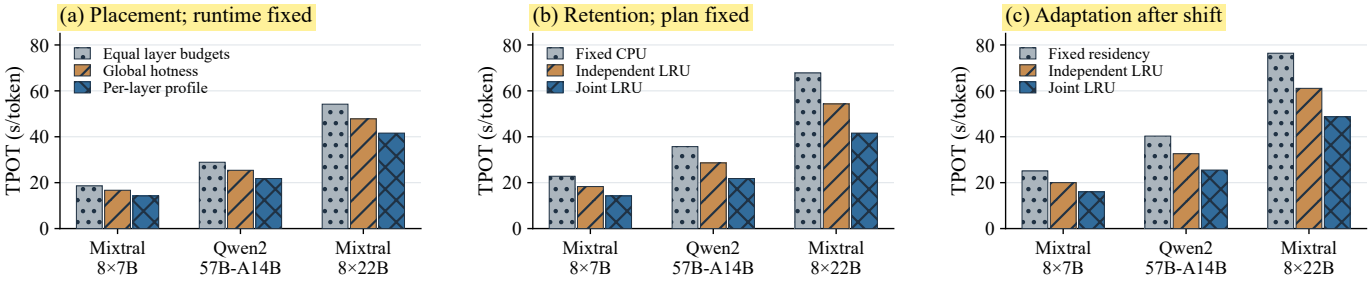


Fig. 8. Component comparisons on three models. (a) Placement with runtime fixed. (b) Retention with initialization fixed. (c) Adaptation after a task switch.

C. Mechanism Attribution

Figure 8 varies one decision in each panel. The placement comparison keeps the runtime rule fixed; retention keeps initialization fixed; adaptation follows a task switch. For Mixtral-8x7B, per-layer placement changes TPOT from 18.6 to 14.2 s/token. Joint retention changes it from 18.3 to 14.2, and online adaptation changes the post-switch value from 25.1 to 16.0. The same comparisons span three models, relating each mechanism to its allocation or request sequence.

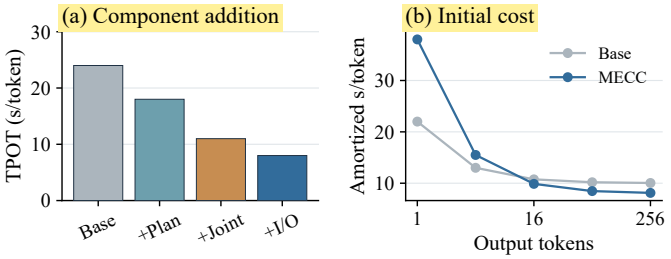


Fig. 9. Ablation and startup. The right panel reports initialization cost per output token plus decode cost.

Adding placement, joint retention, and I/O scheduling gives the TPOT sequence 24, 18, 11, and 8 s/token in Figure 9(a). The final step keeps HDD demand at 20 percent and changes delivery timing. Panel (b) includes initialization through $T_{\text{avg}}(n) = T_{\text{init}}/n + T_{\text{decode}}$. The curves cross at 9 output tokens. Short responses emphasize initial cost; later tokens amortize that cost and expose the decode-time difference.

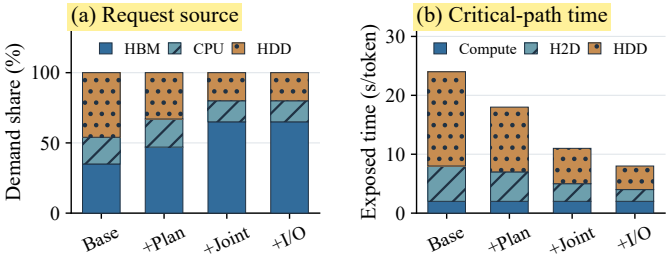


Fig. 10. Source demand and exposed time. The last I/O step preserves the demand fractions while shortening the exposed read/copy path.

Figure 10 decomposes that sequence into request sources and exposed time. Placement and joint retention lower the HDD-demand share from 46 to 20 percent, expanding the fraction served by the resident tiers. The I/O step preserves the source distribution and shortens the delivery components. The time stacks sum to the TPOT values in Figure 9. A request's source and its exposed time describe different parts of execution: host residency avoids an HDD read, and scheduling overlaps remaining transfers with ready work. The two panels locate the effect of each added component in the memory path.

D. Model and Deployment Scaling

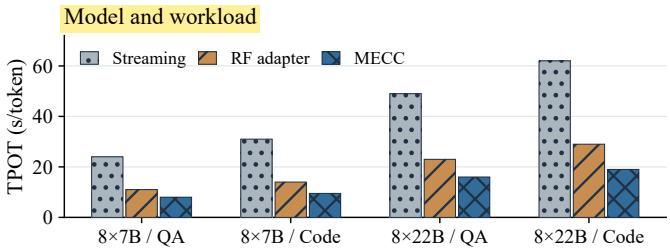


Fig. 11. Model and workload comparison. Mixtral-8x7B and 8x22B with QA/code inputs. Each policy uses the same model and workload within a group.

Figure 11 compares QA and code inputs on two Mixtral sizes. MECC spans 8 to 19 s/token, streaming 24 to 62, and RF 11 to 29. The increase across model sizes appears in each policy. Within each model, QA and code produce different expert demand, changing the weights admitted and delivered during generation. Grouping the policies by model and input keeps those changes visible in the comparison.

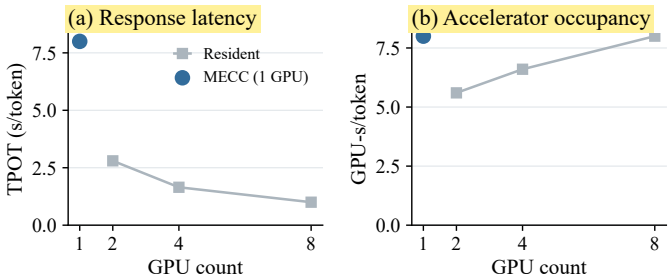


Fig. 12. One-GPU/resident-reference tradeoff. The panels compare single-GPU MECC with resident configurations and report latency alongside GPU occupancy.

The endpoints in Figure 12 pair one GPU at 8 s/token with eight GPUs at 1 s/token. Both use 8 GPU-s/token, with different response times. The two-GPU resident point uses 5.6 GPU-s/token. Plotting the intermediate configurations shows how accelerator provisioning changes both latency and occupancy. The single-GPU point quantifies the latency of supplying weights through the memory hierarchy.

E. Capacity and Workload Sensitivity

Increasing the host allowance from 30 to 60 GB changes MECC TPOT from 14.46 to 0.76 s/token. The reported HDD-demand share changes from 32.2% to zero. At 60 GB, four HBM slots and four CPU slots cover all eight experts per layer. Expert requests then use the two resident tiers. RF takes 13.06 and 0.74 s/token at the same capacity points.

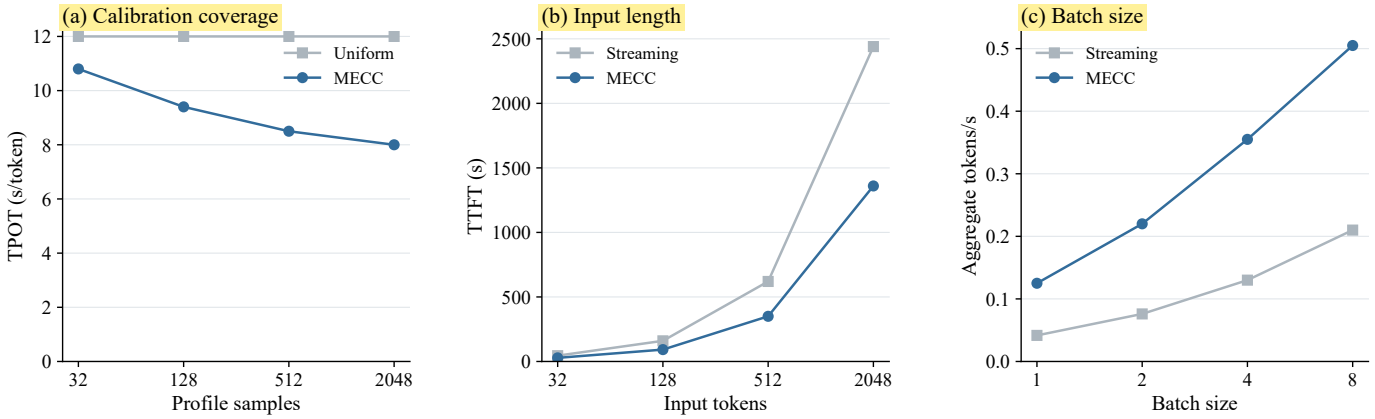


Fig. 13. Profile, input-length, and batch sensitivity. Profile size changes initialization quality, input length changes first-token work, and batch size changes aggregate throughput. The panels vary one workload parameter at a time.

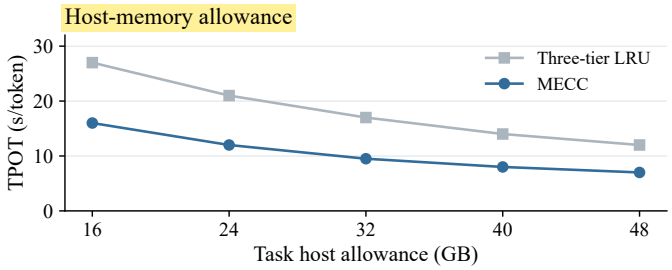


Fig. 14. Host-memory allowance sweep. Five host allowances compare MECC and three-tier LRU within one comparison protocol.

Figure 14 adds the intermediate allowances. MECC changes from 16 to 7 s/token across 16–48 GB, compared with 27 to 12 for three-tier LRU. Increasing host capacity retains more experts after GPU eviction, reducing later cold requests. The curve approaches the resident regime as the combined tiers cover more of each layer's expert set. Capacity thus changes both the number of retained victims and the source used by a later request.

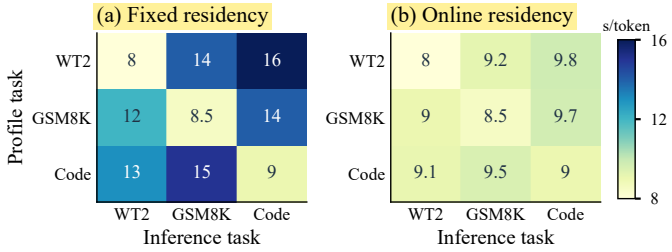


Fig. 15. Profile transfer across tasks. Rows select the calibration task and columns the inference task. Cells are TPOT in s/token; the two panels share a color scale.

The profile-guided curve in Figure 13 falls as calibration size grows, while the uniform control stays flat. Longer prompts increase TTFT; batching increases aggregate token rate. Figure 15 then transfers the calibration profile between WikiText-2, GSM8K, and code inputs. Diagonal cells pair the same task, and off-diagonal cells change the inference task. The fixed cache retains the calibration IDs throughout serving. Online admission instead brings newly selected experts into those slots. Its lower off-diagonal values show the effect of admitting weights selected by the new task.

VII. Related Work

MoE-CORE combines nonuniform device-cache allocation, domain-informed initialization, online replacement, and cross-layer prefetching [13]. Its main path executes router-selected experts, with optional substitution for eligible misses. MECC also uses a profile to initialize residency. It connects the resulting layer capacities to a joint HBM/CPU admission order, carrying GPU victims into host residency during generation.

SSD-LLaMA manages RAM/VRAM caches and pipelines storage reads with weight delivery [14]. It can retain GPU victims in RAM and combines caching with lossless compression and CPU/GPU work partitioning. Edge FlashMoE studies device-cache replacement for on-demand storage access [15]. MECC's retention comparisons use whole FP16 experts with GPU execution, separating the resident state from the weight representation and execution device.

Early expert offloading and SwapMoE expose placement and paging choices [8], [30]. Heterogeneous systems select CPU/GPU execution [10], [31] or pipeline host and device work [11]. Profiling, prediction, and lookahead guide expert delivery in MoE-Infinity, Pro-MoE, Pre-gated MoE, Fate, and ExpertFlow [9], [12], [32]–[34]. Other designs change the cached unit or representation through precision selection [35], [36], expert segments [37], precision slices [38], or staging and compression [39], [40]. These choices affect the bytes and work associated with each expert request.

Klotski and Bevy schedule work across batches or requests [41], [42]; OD-MoE distributes on-demand loading across edge nodes [43]. HeteGen studies heterogeneous dense-model inference [44]. Such schedulers expose work from outside the current gate. MECC's delivery path instead uses the expert work already selected at that gate. Residency controls which of those weights are ready, while the execution schedule controls when their consumers run.

VIII. Conclusion

MECC compiles an initial expert placement and updates it through joint HBM/CPU retention. Per-layer ranking determines the starting residents; promotion and victim demotion carry later reuse across the tier boundary. The delivery path orders reads, copies, and GPU consumption for the original router selections. The A100 comparison reports 7.38 s/token for MECC, alongside source-tier demand and perplexity. Component and workload comparisons relate these outcomes to allocation, admission, and exposed weight delivery.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[6] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[7] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," *arXiv preprint arXiv:2506.04667v3*, 2025.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[10] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[11] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[12] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[13] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[14] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[15] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[16] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[17] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[18] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[19] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[20] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[21] Z. Doucet, R. Sharma, M. de Vos, R. Pires, A.-M. Kermarrec, and O. Balmau, "HarMoEny: Efficient multi-GPU inference of MoE models," *arXiv preprint arXiv:2506.12417v2*, 2025.

[22] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[23] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[24] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[25] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[26] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[27] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[28] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[29] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.

[30] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[31] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[32] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[33] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[34] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[35] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[36] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[37] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[38] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[39] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[40] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[41] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[42] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[43] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.

[44] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.

Appendix A
Experimental Configuration and Figure Data

A. Hardware and Weight Representation

The measured testbed uses one NVIDIA A100 80 GB PCIe. Expert weights use FP16. Dense layers and the router execute on the GPU. The expert store resides on a RAID logical volume reporting rotational media through ROTA=1. The manuscript calls this backing tier HDD. The logical-device record identifies the exposed storage path; the controller's physical-drive inventory is a separate platform record.

Each stored expert contains its gate, up, and down weight tensors. Layer and expert IDs identify the block used by the loader. HBM holds GPU residents. Host memory holds CPU residents, mirrors, and staging tensors. HDD retains the complete expert pool throughout inference.

B. Measured Protocols

The main comparison uses Mixtral-8x7B-Instruct-v0.1, four HBM slots and three CPU slots per layer, and uniform initialization. It processes eight WikiText-2 prompts with 64 input tokens and 64 output tokens. Each policy has one measured run. The planner settings are 50 GB for HBM expert slots and 40 GB for CPU expert slots. Perplexity uses a 1024-token WikiText-2 sample. The same loader and model graph serve the policy comparisons.

The 30/60 GB host-allocation comparison uses four WikiText-2 prompts and 16 output tokens. Its HBM allocation has four slots per layer. At the 60 GB setting, four HBM and four CPU slots cover the eight experts in each layer. The paper presents these points as a separate capacity comparison. The shorter protocol is retained with the corresponding numbers.

C. Policy Definitions

Streaming loads selected experts through the shared delivery path. Fixed CPU keeps an initial host set and uses GPU replacement. Independent caching updates HBM and CPU recency separately. MECC couples GPU promotion to host admission of the GPU victim. These controls share the model and expert store.

The RF adapter derives its public recency and frequency features from edge FlashMoE. It uses HBM LRU and a CPU score that combines inverse recency with normalized access count. Equal deterministic weights combine the two score terms. The paper labels it RF adapter to identify this mechanism implementation. CPU-compute evaluates selected expert work on the host. The I/O variant supplies a separate delivery control.

For candidate expert e, the RF score is $0.5/d_e + 0.5 f_e / f_{\max}$. Here d_e is recency distance, f_e is cumulative access count, and $f_{\max}$ normalizes frequency. The host cache removes the candidate with the lowest score. Expert ID breaks score ties. The access clock advances on each request.

D. Timing and Demand Counters

TTFT denotes time to first token. TPOT denotes time per output token during decode. Accelerator occupancy is GPU count multiplied by TPOT, reported in GPU-s/token. It describes accelerator provisioning over response time. The paper reports that product alongside latency.

The measured HDD-demand shares aggregate all instrumented phases. They count classified expert accesses. An expert installed by prefetch is classified by its resident tier at consumption. Read and copy traffic use different byte counters. Initialization, warmup, prefill, decode, and PPL are distinct execution windows. The reported demand percentages retain the aggregate window used by the source runs. The short-window correctness diagnostic compares the streaming and MECC paths through routing, logits, gates, and generated tokens. The reported PPL values use the stated WikiText-2 sample.

E. Figure Data Annotations

Yellow-highlighted numerical values and panel titles identify authored design targets. They are unmeasured values used to specify the illustrated comparisons. The unhighlighted A100 results come from committed run records. Thus the source of a panel remains distinct from the mechanism illustrated beside it.

The Introduction comparison and single-A100 runtime figure use measured results. In the Motivation figure, panel (b) uses measured aggregate demand; panels (a,c) use design arrays. Panels (d–f), the framework, and the method diagrams describe mechanisms. Their expert IDs, slot counts, and schedules serve as examples of the corresponding operations.

The component, startup, traffic/exposure, cross-model, GPU-occupancy, quota, workload-sensitivity, and profile-transfer plots use design targets. The model labels name the configurations to compare. QA and Code identify the workload groups in those arrays. WikiText-2 and GSM8K identify language-modeling and arithmetic inputs. The Code label identifies the code-input group. These arrays do not add measured model or benchmark runs.

The plotted startup example uses $T_{\text{avg}}(n) = T_{\text{init}}/n + T_{\text{decode}}$. Its crossing at nine output tokens follows from the supplied initial and decode costs. The source-demand stacks sum to 100%. The exposed-time stacks sum to the component TPOT values. These checks align the panels within the design example.

F. Memory and Execution Accounting

Planner allowances count primary expert slots. The implementation also retains host mirrors for GPU residents. Those mirrors consume host storage and supply later demotions. Staging and temporary copies add allocations during delivery. Physical accounting therefore tracks primary residency, mirrors, and staging separately from the slot plan.

The compiler library accepts per-layer skew parameters and ranked IDs. The measured main run uses uniform initialization. The non-uniform e2e CLI branch generates layer skew parameters; a calibrated profile is an explicit compiler input. Prefill submits read futures for the selected expert batch. Decode loads experts on demand. Copy-stream synchronization orders CUDA weight consumption, and consuming-stream tracking preserves tensor storage through kernel execution.

G. Plan and Request Records

A compiled plan stores the model identifier, layer count, experts per layer, expert size, per-layer capacities, and ranked IDs. The capacity arrays define the HBM and CPU cuts. The placement array maps ranks back to original expert identities. Cache initialization consumes both arrays.

H. Source Records

Committed per-run JSON files supply the A100 measurements. The measured-figure audit lists the run IDs and field mappings. The two design-array files supply the highlighted figures. Their source manifest records field names and SHA-256 hashes. The revision preserves those arrays and the measured plots. Each archived manuscript version includes its sources and an artifact hash record.